PROGRAMA ACADÉMICO TEÓRICO–PRÁCTICO (36 HORAS)

Aprendizaje por Refuerzo

De la Probabilidad a las Redes Neuronales Profundas



Profesor Especialista en Aprendizaje por Refuerzo

Metodología: Lápiz, Papel y Código · Caso Conductor: OptiBot

Edición 2026

Prólogo: Fundamentos del Aprendizaje por Refuerzo

El aprendizaje por refuerzo (*Reinforcement Learning*, o RL) constituye uno de los paradigmas más representativos y potentes de la inteligencia artificial contemporánea. A diferencia del aprendizaje supervisado clásico, en el cual los modelos dependen de conjuntos de datos previamente etiquetados por un instructor externo, el aprendizaje por refuerzo modela el proceso mediante el cual un agente autónomo aprende a tomar decisiones óptimas interactuando directamente con su entorno mediante ensayo, error y señales escalares de recompensa.

Alcance y Estructura del Texto

Este texto ha sido estructurado con el objetivo de ofrecer una exposición progresiva, autosuficiente y matemáticamente transparente para estudiantes e ingenieros que posean conocimientos elementales de programación en Python y álgebra básica.

No se asumen conocimientos avanzados previos en probabilidad axiomática, procesos estocásticos ni teoría de control moderno. Cada concepto fundamental se introduce a partir de sus principios elementales, vinculando tres componentes en cada etapa:

1. **Intuición conceptual:** El planteamiento físico o lógico del problema de toma de decisiones.
2. **Deducción y cálculo manual paso a paso:** El desarrollo explícito de los cálculos numéricos y algebraicos en casos reducidos, permitiendo comprender la mecánica exacta de las ecuaciones.
3. **Implementación algorítmica:** La traducción directa de las fórmulas matemáticas a implementaciones ejecutables y reproducibles en Python (NumPy y PyTorch).

El Hilo Conductor: El Sistema Logístico OptiBot

Con el fin de mantener una coherencia conceptual continua a lo largo de toda la obra, el desarrollo teórico se articula en torno a un caso de aplicación industrial representativo: **OptiBot**, un robot móvil autónomo destinado a la gestión de inventario y transporte de carga en un almacén inteligente.

- Los Capítulos 1 y 2 abordan la cuantificación de la incertidumbre estocástica y la toma de decisiones evaluativas bajo el dilema de exploración frente a explotación (Bandidos Multi-Brazo).
- Los Capítulos 3 y 4 formalizan la evolución dinámica del entorno mediante Cadenas de Markov y Procesos de Decisión de Markov (MDP), deduciendo analíticamente las Ecuaciones de Bellman bajo modelos analíticos conocidos.

- El Capítulo 5 desarrolla el **Aprendizaje por Refuerzo Tradicional**, presentando el salto al aprendizaje libre de modelo mediante Diferencia Temporal y deduciendo exhaustivamente el algoritmo tabular **Q-Learning**.
- El Capítulo 6 culmina con el **Aprendizaje por Refuerzo Profundo**, abordando la transición hacia espacios sensoriales continuos mediante Redes Neuronales Profundas y el algoritmo **Deep Q-Learning (DQN)**.

El texto invita al estudiante a realizar activamente los cálculos manuales y experimentos computacionales propuestos en cada sección, consolidando una comprensión profunda y operativa del aprendizaje por refuerzo.

Índice general

Prólogo: Fundamentos del Aprendizaje por Refuerzo **1**

1. Aprendizaje por Refuerzo Tradicional: Algoritmo Q-Learning **4**
1.1. De las Ecuaciones Analíticas al Aprendizaje sin Modelo (Model-Free) 4
1.2. El Laberinto de OptiBot: Un Almacén en Cuadrícula de 3×3 4
1.3. ¿Qué es exactamente la Tabla Q? La Memoria de Navegación 6
1.3.1. Estructura de la Tabla Q 6
1.3.2. La Regla de Decisión: Elegir la Flecha con Mayor Puntuación 6
1.4. La Ecuación de Q-Learning Explicada Paso a Paso 7
1.5. El Dilema: ¿Cómo Explorar el Laberinto? (Política ε-Greedy) 7
1.6. Cálculo Manual Paso a Paso: Siguiendo a OptiBot en el Laberinto 8
1.7. Evolución a Largo Plazo: Cómo los Ceros se Transforman en Valores Negativos . . 14
1.7.1. El auge de los pasillos interiores: Descubrimiento de dos rutas óptimas . . . 14
1.8. Visualización Gráfica: La Política Aprendida en la Cuadrícula 15
1.9. Implementación en Python: El Laberinto de OptiBot 16
1.10. La Gran Limitación: La Maldición de la Dimensionalidad 16

2. Aprendizaje por Refuerzo Profundo: Deep Q-Learning (DQN) **17**
2.1. De Tablas Discretas a Aproximadores de Funciones Continuos 17
2.2. La Tríada Mortal: Por qué el RL Profundo Ingenuo Fracasa 18
2.2.1. Las Dos Patologías Destructivas del Aprendizaje 18
2.3. La Revolución de DQN: Los Dos Mecanismos Estabilizadores (DeepMind, 2015) . 19
2.3.1. 1. Buffer de Repetición de Experiencias (*Experience Replay*) 19
2.3.2. 2. Red Objetivo Congelada (*Target Network*) 19
2.4. Función de Pérdida y Descenso de Gradiente 20
2.4.1. Pérdida de Huber (Smooth L1 Loss) 20
2.4.2. El Enfoque del Semi-Gradiente 20
2.5. Cálculo Numérico Paso a Paso: Flujo de Tensores en OptiBot 20
2.6. Double DQN (DDQN): Eliminación del Sesgo de Sobreestimación 21
2.7. Implementación Modular en PyTorch: OptiBot DQN 22
2.8. Comparativa Global: De las Tablas a las Redes Neuronales 22

1

Aprendizaje por Refuerzo Tradicional: Algoritmo Q-Learning

1.1 De las Ecuaciones Analíticas al Aprendizaje sin Modelo (Model-Free)

En el Capítulo **??**, formalizamos los Procesos de Decisión de Markov (MDP) y descubrimos cómo calcular la política óptima mediante la **Ecuación de Bellman** y el algoritmo de evaluación iterativa. Sin embargo, en aquel escenario asumíamos una condición ideal: conocíamos de antemano el **modelo completo del entorno**, es decir, las tablas exactas de probabilidad de transición $\mathcal{P}^a_{ss'}$ y las funciones analíticas de recompensa $\mathcal{R}(s, a)$.

En el mundo físico real, esa suposición casi nunca se cumple:

- Un robot móvil no dispone de un manual con las probabilidades exactas de que sus ruedas patinen o de que una compuerta se trabe.
- Modelar matemáticamente cada interacción física, fricción y obstáculo imprevisto resulta inviable.
- El agente debe enfrentarse a lo desconocido: solo puede descubrir qué acciones son seguras y cuáles son perjudiciales **interactuando directamente con el entorno mediante prueba y error**.

Este es el territorio del **Aprendizaje por Refuerzo Tradicional**: el paradigma **libre de modelo** (*Model-Free RL*). El agente no asume conocer cómo funciona el mundo en teoría; en su lugar, **aprende observando las consecuencias reales de sus decisiones**, recopilando experiencias paso a paso y corrigiendo su estrategia a través del algoritmo fundamental de Watkins (1989): **Q-Learning**.

1.2 El Laberinto de OptiBot: Un Almacén en Cuadrícula de 3×3

Para comprender el funcionamiento de Q-Learning sin perderse en fórmulas abstractas, consideremos el problema arquetípico del aprendizaje por refuerzo: el **almacén inteligente modelado como una cuadrícula o laberinto de** 3×3 (Figura 1.1).

Las reglas de este entorno son transparentes y directas:

1. **Espacio de Estados ($\mathcal{S}$):** Hay 9 casillas numeradas del S_1 al S_9.

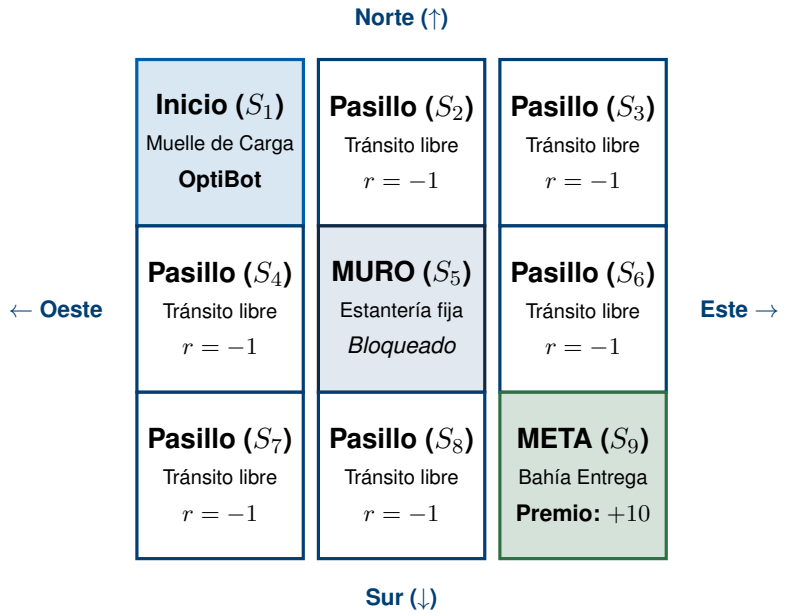


Figura 1.1: El entorno del laberinto logístico de OptiBot (3×3). El robot parte del Muelle (S_1) y debe alcanzar la Meta (S_9) esquivando la estantería central bloqueada (S_5). Cada movimiento ordinario descuenta -1 de recompensa, incentivando descubrir la ruta más rápida.

- S_1 es la casilla de **Inicio** donde parte OptiBot.
- S_5 es un **Muro** (una estantería fija impenetrable): el robot no puede ingresar en ella.
- S_9 es la **Meta** (Bahía de Entrega final): es un estado terminal que concede una bonificación de $+10$ puntos al llegar y concluye el episodio.

2. **Acciones disponibles ($\mathcal{A}$):** En cualquier casilla, OptiBot puede intentar moverse en cuatro direcciones cardinales:

$$\mathcal{A} = \{\text{Norte } (\uparrow), \text{ Sur } (\downarrow), \text{ Este } (\rightarrow), \text{ Oeste } (\leftarrow)\}$$

3. **Dinámica, Paredes y Salida del Laberinto:**
   - **Muro Central (S_5):** Es una estantería fija impenetrable. Si OptiBot intenta ingresar a S_5, rebota contra el obstáculo y permanece en su misma casilla, sufriendo la penalización normal de paso ($R = -1.0$).
   - **Límites del Laberinto (Game Over):** Si OptiBot intenta moverse hacia fuera de los límites perimetrales de la cuadrícula (por ejemplo, avanzar al Norte o al Oeste desde S_1), el robot cae fuera del recinto de trabajo. En este caso, el entorno le aplica una severa penalización de $R = -10.0$ y el episodio concluye de inmediato (*Game Over*, estado terminal con valor futuro nulo).

4. **Señal de Recompensa (R):**
   - Cada paso ordinario por los pasillos cuesta $R = -1.0$ (penalización por tiempo y consumo de batería).
   - Intentar salirse de los límites del laberinto cuesta $R = -10.0$ y causa *Game Over* (fin de episodio con penalización severa).
   - Ingresar a la Meta (S_9) otorga una gran recompensa de $R = +10.0$ (fin de episodio exitoso).

El objetivo de OptiBot es encontrar la **secuencia más corta de movimientos** que lo conduzca a la Meta (S_9) esquivando el muro central (S_5) y acumulando la máxima puntuación posible, **sin conocer de antemano el mapa ni las recompensas**.

1.3 ¿Qué es exactamente la Tabla Q? La Memoria de Navegación

Imaginemos que OptiBot lleva consigo una libreta de notas con una cuadrícula dibujada. A esa libreta la llamamos la **Tabla Q**.

La letra **Q** proviene del término inglés *Quality* (Calidad). Cada casilla de la libreta le asigna una puntuación a la pregunta más importante del aprendizaje:

> *«Si estoy en la casilla s y doy un paso en la dirección a, ¿qué tan buena idea es ese movimiento para mi futuro?»*

1.3.1 Estructura de la Tabla Q

La Tabla Q es simplemente una matriz bidimensional:

- Las **filas** representan las casillas del almacén ($S_1, S_2, \ldots, S_9$).
- Las **columnas** representan las 4 direcciones posibles ($\uparrow, \downarrow, \rightarrow, \leftarrow$).
- El valor dentro de la celda $Q(s, a)$ es un número decimal que estima el retorno acumulado total esperado.

Estado (Casilla)	Norte ($\uparrow$)	Sur ($\downarrow$)	Este ($\rightarrow$)	Oeste ($\leftarrow$)
S_1 (Inicio)	0.00	0.00	0.00	0.00
S_2 (Pasillo)	0.00	0.00	0.00	0.00
S_3 (Pasillo)	0.00	0.00	0.00	0.00
S_4 (Pasillo)	0.00	0.00	0.00	0.00
S_5 (Muro)	—	—	—	—
S_6 (Pasillo)	0.00	0.00	0.00	0.00
S_7 (Pasillo)	0.00	0.00	0.00	0.00
S_8 (Pasillo)	0.00	0.00	0.00	0.00
S_9 (Meta)	0.00	0.00	0.00	0.00

Cuadro 1.1: La Tabla Q inicial de OptiBot. Al comenzar, todas las celdas valen cero: el robot es una "página en blanco" que desconoce dónde está la meta y dónde están los muros.

1.3.2 La Regla de Decisión: Elegir la Flecha con Mayor Puntuación

Una vez que la tabla tiene números aprendidos, la toma de decisiones es directa: el robot consulta la fila de la casilla s en la que se encuentra y selecciona la dirección con la mayor puntuación. Esta regla formaliza la **política codiciosa** (*greedy policy*):

$$\pi(s) = \arg\max_{a \in \mathcal{A}} Q(s, a) \tag{1.1}$$

En palabras sencillas, esta fórmula representa la regla de navegación de OptiBot: el operador $\arg\max$ no se queda con el puntaje numérico, sino que extrae la **acción física concreta** (Norte,

Sur, Este u Oeste) que produce el valor $Q(s, a)$ más alto en dicha casilla. ¡Nótese la enorme ventaja frente al capítulo anterior! No se requieren probabilidades de transición, ni integrales, ni sumatorias. Basta con mirar la fila y tomar el máximo.

1.4 La Ecuación de Q-Learning Explicada Paso a Paso

¿Cómo hace el robot para llenar esa tabla con números correctos a partir de la experiencia directa? Lo hace aplicando en cada paso la siguiente ecuación de actualización:

$$\underbrace{Q(S_t, A_t)}_{\text{Nuevo valor}} \leftarrow \underbrace{Q(S_t, A_t)}_{\text{Valor viejo}} + \alpha \cdot \Big[\underbrace{\overbrace{R_{t+1}}^{\text{Recompensa inmediata}} + \gamma \overbrace{\max_{a'} Q(S_{t+1}, a')}^{\text{Mejor valor del destino}}}_{\text{Realidad experimentada (Objetivo Bellman)}} - \underbrace{Q(S_t, A_t)}_{\text{Lo que creía antes}} \Big] \tag{1.2}$$

Desglosemos cada pieza de esta fórmula con total transparencia:

1. **¿Dónde estoy y qué hice? (S_t, A_t):** Es la casilla actual donde estaba el robot (S_t) y la dirección en la que decidió moverse (A_t). La celda $Q(S_t, A_t)$ es la que vamos a corregir en la tabla.
2. **La Recompensa Inmediata (R_{t+1}):** Es la señal concreta que el entorno devuelve inmediatamente tras dar el paso (por ejemplo, -1 por el costo de energía, o $+10$ si aterrizó en la Meta).
3. **La Mirada al Futuro ($\max_{a'} Q(S_{t+1}, a')$):** Cuando OptiBot llega a la nueva casilla (S_{t+1}), abre su libreta en la fila de esa nueva casilla y se pregunta:

   > *«Desde esta nueva casilla donde aterricé, ¿cuál es la mejor salida posible que promete mi tabla?»*

   Toma el número más alto de esa fila y lo multiplica por γ (el factor de descuento, típicamente 0.90), que refleja que las recompensas futuras valen un poco menos que las inmediatas.
4. **La Sorpresa o Error TD (δ_t):** Compara la realidad experimentada (la recompensa recibida más el mejor futuro prometido) contra lo que antes creía que valía esa acción:

$$\delta_t = \Big[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') \Big] - Q(S_t, A_t) \tag{1.3}$$

   - Si $\delta_t > 0$, el resultado fue **mejor de lo que esperaba** (¡sorpresa agradable!), por lo que la puntuación de esa casilla sube.
   - Si $\delta_t < 0$, el resultado fue **peor de lo que esperaba** (costo inesperado o golpe contra un muro), por lo que la puntuación baja.
5. **La Tasa de Aprendizaje (α):** Regula qué tan rápido aprende el robot (habitualmente $\alpha = 0.2$ o 0.5). Evita que el agente reaccione de forma exagerada ante un único resultado aislado, promediando suavemente las experiencias.

1.5 El Dilema: ¿Cómo Explorar el Laberinto? (Política ε-Greedy)

Si al principio todas las casillas valen 0.00, o si OptiBot descubre un camino mediocre y siempre elige la acción con el valor más alto actual, el robot se quedará atrapado repitiendo el mismo camino tonto y jamás descubrirá dónde está la Meta. Para solucionar esto, se utiliza la estrategia ε**-greedy** (*épsilon-codiciosa*):

- Con probabilidad $1-\varepsilon$ (por ejemplo, el 80 % de las veces), el robot **explota**: elige la dirección que tenga el valor Q más alto en su tabla.
- Con probabilidad ε (por ejemplo, el 20 % de las veces), el robot **explora**: tira una moneda al aire y elige una dirección completamente al azar para explorar rincones desconocidos.

Al principio del entrenamiento, se fija $\varepsilon = 1.0$ (exploración total). Conforme el robot completa cientos de misiones y la tabla se va llenando de números fiables, ε decae gradualmente hasta 0.01, pasando a un modo donde casi siempre utiliza su conocimiento experto aprendido.

1.6 Cálculo Manual Paso a Paso: Siguiendo a OptiBot en el Laberinto

Para ver cómo funciona la matemática con números exactos, sigamos el recorrido de OptiBot paso a paso con lápiz y papel.

Parámetros del ejercicio:

- Tasa de aprendizaje: $\alpha = 0.50$ (para simplificar los cálculos a números exactos).
- Factor de descuento: $\gamma = 0.90$.
- Recompensa de paso normal por pasillo: $R = -1.0$.
- Penalización por salirse del laberinto (*Game Over*): $R = -10.0$ (estado terminal con valor futuro 0).
- Recompensa en la Meta (S_9): $R = +10.0$ (estado terminal con valor futuro 0).

Inicialmente, todas las celdas de la Tabla Q valen 0.00.

Episodio 1 – Paso 1: El primer paso en el laberinto

1. OptiBot se encuentra en el Inicio (S_1).
2. Elige al azar la acción **Este ($\rightarrow$)**.
3. Se desplaza y aterriza en la casilla S_2. Recibe una recompensa de paso $R_1 = -1$.
4. Mira la fila del estado destino S_2 en su tabla: como todo vale cero, el máximo futuro es:
$$\max_{a'} Q(S_2, a') = \max(0, 0, 0, 0) = \mathbf{0.0}$$
5. Calcula el Objetivo de Bellman:
$$\text{Objetivo} = R_1 + \gamma \max_{a'} Q(S_2, a') = -1 + (0.90 \times 0.0) = \mathbf{-1.0}$$
6. Calcula el Error TD:
$$\delta = \text{Objetivo} - Q(S_1, \rightarrow) = -1.0 - 0.0 = \mathbf{-1.0}$$
7. Actualiza la celda $Q(S_1, \rightarrow)$:
$$Q_{\text{nuevo}}(S_1, \rightarrow) = 0.0 + 0.50 \times (-1.0) = \mathbf{-0.50}$$

Interpretación: Ahora la tabla sabe que moverse al Este desde S_1 tiene un valor de -0.50.

Episodio 1 – Paso 2: Avanzando por el laberinto

1. OptiBot está en S_2 y elige moverse hacia el **Este (→)**.
2. Aterriza en la esquina superior derecha (S_3) y recibe $R_2 = -1$.
3. El valor futuro en S_3 es cero. Actualizamos:

$$Q_{\text{nuevo}}(S_2, \rightarrow) = 0.0 + 0.50 \times [-1 + 0.9(0) - 0] = \mathbf{-0.50}$$

Episodio 1 – Paso 3: Bajando hacia el sur

1. Desde S_3, el robot elige moverse hacia el **Sur (↓)**.
2. Aterriza en la casilla S_6 (a la derecha del muro S_5) y recibe $R_3 = -1$.
3. Actualizamos de forma análoga:

$$Q_{\text{nuevo}}(S_3, \downarrow) = 0.0 + 0.50 \times [-1 + 0.9(0) - 0] = \mathbf{-0.50}$$

Episodio 1 – Paso 4: ¡El gran momento, llegada a la Meta!

OptiBot está en S_6 (justo arriba de la Meta S_9):

1. Elige moverse hacia el **Sur (↓)**.
2. ¡Aterriza en la Meta (S_9)! El entorno le entrega una recompensa masiva: $R_4 = +\mathbf{10.0}$.
3. Como S_9 es un estado terminal que concluye la misión, el valor futuro a partir de la meta es estrictamente cero ($\max_{a'} Q(S_9, a') = 0$).
4. Calculamos el Objetivo de Bellman:

$$\text{Objetivo} = R_4 + \gamma(0) = +\mathbf{10.0}$$

5. Calculamos el Error TD:

$$\delta = \text{Objetivo} - Q(S_6, \downarrow) = +10.0 - 0.0 = +\mathbf{10.0}$$

6. Actualizamos la celda $Q(S_6, \downarrow)$:

$$Q_{\text{nuevo}}(S_6, \downarrow) = 0.0 + 0.50 \times (+10.0) = +\mathbf{5.00}$$

¡Momento revelador!: La celda que conduce directamente a la Meta $(S_6, \downarrow)$ no tiene un valor negativo; ¡posee un enorme $+5.00$ positivo! La Meta ha encendido su primera señal luminosa en la memoria del agente.
Al finalizar este primer episodio, la Tabla Q queda actualizada como se muestra a continuación:

Estado (Casilla)	Norte (↑)	Sur (↓)	Este (→)	Oeste (←)
S_1 (Inicio)	0.00	0.00	$\mathbf{-0.50}$	0.00
S_2 (Pasillo)	0.00	0.00	$\mathbf{-0.50}$	0.00
S_3 (Pasillo)	0.00	$\mathbf{-0.50}$	0.00	0.00
S_4 (Pasillo)	0.00	0.00	0.00	0.00
S_5 (Muro)	—	—	—	—
S_6 (Pasillo)	0.00	$\mathbf{+5.00}$	0.00	0.00
S_7 (Pasillo)	0.00	0.00	0.00	0.00
S_8 (Pasillo)	0.00	0.00	0.00	0.00
S_9 (Meta)	0.00	0.00	0.00	0.00

Nota: En negrita se destacan las cuatro únicas celdas modificadas tras recorrer la trayectoria $S_1 \xrightarrow{\rightarrow} S_2 \xrightarrow{\rightarrow} S_3 \xrightarrow{\downarrow} S_6 \xrightarrow{\downarrow} S_9$. Las demás acciones y casillas aún no han sido exploradas y permanecen en 0.00.

Episodio 2 – Paso 1: Selección basada en la política derivada de Q (S_1)

OptiBot vuelve a colocarse en la casilla de Inicio (S_1) con su Tabla Q actualizada del Episodio 1.
Siguiendo el algoritmo estricto de Q-Learning, el agente debe elegir la acción A usando una política derivada de Q (en este caso, la regla codiciosa $\arg\max_a Q(S_1, a)$). Consultamos la fila de S_1 en la tabla:

$$Q(S_1, \uparrow) = 0.00, \quad Q(S_1, \downarrow) = 0.00, \quad Q(S_1, \rightarrow) = \mathbf{-0.50}, \quad Q(S_1, \leftarrow) = 0.00$$

Decisión crucial y la paradoja de los ceros: El valor más alto en la fila es $\max_a Q(S_1, a) = \mathbf{0.00}$, correspondiente a $\{\uparrow, \downarrow, \leftarrow\}$. La acción Este ($\rightarrow$), penalizada con -0.50 tras el paso inicial del Episodio 1, **no es la opción más alta** y queda descartada por la regla codiciosa.
¿Por qué Norte y Oeste tienen 0.00, pareciendo erróneamente mejores que Este (-0.50)? Porque son acciones aún no exploradas. En Q-Learning, los ceros iniciales representan una ausencia total de experiencia (optimismo inicial). Si OptiBot intentara en este paso moverse al Norte o al Oeste, se saldría de la cuadrícula, sufriendo la penalización terminal catastrófica de $R = -10.0$ (*Game Over*). En una sola actualización de Bellman, dicha acción se desplomaría a:

$$Q_{\text{nuevo}}(S_1, \uparrow) = 0.00 + 0.50 \times [-10.0 + \gamma(0) - 0.00] = \mathbf{-5.00}$$

Por tanto, los ceros en los bordes no son valores ventajosos: son una ilusión transitoria que en cuanto se explora se transforma en números fuertemente negativos (-5.00 y con más visitas hasta -10.00).
Desempatando entre las opciones con valor máximo (0.00) en favor de la dirección transitable que no causa Game Over, OptiBot selecciona la acción: **Sur ($\downarrow$)**.

1. Selecciona la acción con mayor valor transitable: **Sur ($\downarrow$)**.
2. Aterriza en la casilla S_4 y recibe la penalización de paso $R_1 = -1.0$.
3. Consulta la fila de la casilla destino S_4 en su tabla. Como aún no ha sido explorada, todas sus acciones valen 0.00:

$$\max_{a'} Q(S_4, a') = \max(0.00,\ 0.00,\ 0.00,\ 0.00) = \mathbf{0.00}$$

4. Calcula el Objetivo de Bellman:

$$\text{Objetivo} = R_1 + \gamma \max_{a'} Q(S_4, a') = -1.0 + (0.90 \times 0.00) = \mathbf{-1.00}$$

5. Calcula el Error TD:

$$\delta = \text{Objetivo} - Q(S_1, \downarrow) = -1.00 - 0.00 = \mathbf{-1.00}$$

6. Actualiza la celda $Q(S_1, \downarrow)$:

$$Q_{\text{nuevo}}(S_1, \downarrow) = 0.00 + 0.50 \times (-1.00) = \mathbf{-0.50}$$

Episodio 2 – Paso 2: Explorando el pasillo inferior (S_4)

OptiBot se encuentra en S_4:

1. Todas las acciones en S_4 valen 0.00 ($\max_a Q(S_4, a) = 0.00$). Como hacia el Este chocaría con el Muro (S_5), Norte retrocedería y Oeste golpea la pared perimetral, desempata eligiendo la opción máxima hacia la casilla transitable: **Sur (↓)**.
2. Aterriza en la casilla S_7 y recibe la penalización de paso $R_2 = -1.0$.
3. Inspecciona la fila de S_7 en su tabla, donde todas las acciones aún valen 0.00:

$$\max_{a'} Q(S_7, a') = \max(0.00,\ 0.00,\ 0.00,\ 0.00) = \mathbf{0.00}$$

4. Calcula el Objetivo de Bellman:

$$\text{Objetivo} = R_2 + \gamma \max_{a'} Q(S_7, a') = -1.0 + (0.90 \times 0.00) = \mathbf{-1.00}$$

5. Calcula el Error TD:

$$\delta = \text{Objetivo} - Q(S_4, \downarrow) = -1.00 - 0.00 = \mathbf{-1.00}$$

6. Actualiza la celda $Q(S_4, \downarrow)$:

$$Q_{\text{nuevo}}(S_4, \downarrow) = 0.00 + 0.50 \times (-1.00) = \mathbf{-0.50}$$

Episodio 2 – Paso 3: Girando hacia la Meta (S_7)

OptiBot está en la esquina inferior izquierda (S_7):

1. En S_7, todas las acciones valen 0.00 ($\max_a Q(S_7, a) = 0.00$). Desempatando hacia la salida abierta del pasillo, elige **Este (→)**.
2. Aterriza en S_8 recibiendo la penalización de paso $R_3 = -1.0$.
3. Al consultar la fila de la casilla destino S_8, todas sus acciones aún valen 0.00:

$$\max_{a'} Q(S_8, a') = \max(0.00,\ 0.00,\ 0.00,\ 0.00) = \mathbf{0.00}$$

4. Calcula el Objetivo de Bellman:

$$\text{Objetivo} = R_3 + \gamma \max_{a'} Q(S_8, a') = -1.0 + (0.90 \times 0.00) = \mathbf{-1.00}$$

5. Calcula el Error TD:

$$\delta = \text{Objetivo} - Q(S_7, \rightarrow) = -1.00 - 0.00 = \mathbf{-1.00}$$

6. Actualiza la celda $Q(S_7, \rightarrow)$:

$$Q_{\text{nuevo}}(S_7, \rightarrow) = 0.00 + 0.50 \times (-1.00) = \mathbf{-0.50}$$

Episodio 2 – Paso 4: Descubrimiento de la Meta por el Sur (S_8)

OptiBot está en S_8, a la puerta sur de la Meta:

1. Todas las acciones en S_8 valen 0.00 ($\max_a Q(S_8, a) = 0.00$). Elige moverse al **Este (→)**.
2. ¡Aterriza en la Meta (S_9)! Recibe la recompensa masiva $R_4 = +\mathbf{10.0}$.
3. Como S_9 es el estado terminal, el valor futuro es estrictamente cero ($\max_{a'} Q(S_9, a') = 0$).
4. Calcula el Objetivo de Bellman:

$$\text{Objetivo} = R_4 + \gamma(0) = +\mathbf{10.00}$$

5. Calcula el Error TD:

$$\delta = \text{Objetivo} - Q(S_8, \rightarrow) = +10.00 - 0.00 = +\mathbf{10.00}$$

6. Actualiza la celda $Q(S_8, \rightarrow)$:

$$Q_{\text{nuevo}}(S_8, \rightarrow) = 0.00 + 0.50 \times (+10.00) = +\mathbf{5.00}$$

Al finalizar el segundo episodio, la Tabla Q queda actualizada con las dos rutas simétricas descubiertas:

Estado (Casilla)	Norte (↑)	Sur (↓)	Este (→)	Oeste (←)
S_1 (Inicio)	0.00	$\mathbf{-0.50}$	-0.50	0.00
S_2 (Pasillo)	0.00	0.00	-0.50	0.00
S_3 (Pasillo)	0.00	-0.50	0.00	0.00
S_4 (Pasillo)	0.00	$\mathbf{-0.50}$	0.00	0.00
S_5 (Muro)	—	—	—	—
S_6 (Pasillo)	0.00	$+5.00$	0.00	0.00
S_7 (Pasillo)	0.00	0.00	$\mathbf{-0.50}$	0.00
S_8 (Pasillo)	0.00	0.00	$\mathbf{+5.00}$	0.00
S_9 (Meta)	0.00	0.00	0.00	0.00

Interpretación del Episodio 2: El robot descubrió la segunda ruta óptima hacia la meta. Ahora ambos accesos inmediatos a S_9 ($Q(S_6, \downarrow) = +5.00$ por el norte y $Q(S_8, \rightarrow) = +5.00$ por el sur) poseen valores positivos idénticos.

Episodio 3: ¡La propagación de la recompensa hacia atrás!

En el tercer episodio, OptiBot vuelve a situarse en el Inicio (S_1). Ahora, las dos salidas exploradas están empatadas en la cima de las opciones viables ($Q(S_1, \rightarrow) = -0.50$ y $Q(S_1, \downarrow) = -0.50$). Al desempatar hacia el corredor superior (**Este**), observemos lo que ocurre en cada paso:

1. **Paso 1 ($S_1 \rightarrow S_2$):** Se mueve al Este. Con $R_1 = -1.0$ y $\max_{a'} Q(S_2, a') = 0.00$:

$$\delta = -1.00 - (-0.50) = -0.50 \implies Q_{\text{nuevo}}(S_1, \rightarrow) = -0.50 + 0.50(-0.50) = -\mathbf{0.75}$$

2. **Paso 2 ($S_2 \to S_3$):** Avanza al Este. Con $R_2 = -1.0$ y $\max_{a'} Q(S_3, a') = 0.00$:

$$\delta = -1.00 - (-0.50) = -0.50 \implies Q_{\text{nuevo}}(S_2, \to) = -0.50 + 0.50(-0.50) = \mathbf{-0.75}$$

3. **Paso 3 ($S_3 \to S_6$, ¡El salto de valor!):** OptiBot está en S_3 y elige descender al **Sur (↓)**. Aterriza en S_6 recibiendo $R_3 = -1.0$.

   Al consultar la fila del estado destino S_6, ¡ocurre el fenómeno clave!: la casilla hacia el Sur ya tiene el valor positivo descubierto en el Episodio 1:

$$\max_{a'} Q(S_6, a') = \max(0.00,\ \mathbf{+5.00},\ 0.00,\ 0.00) = \mathbf{+5.00}$$

   El Objetivo de Bellman para S_3 se beneficia directamente de esta señal:

$$\text{Objetivo} = R_3 + \gamma \max_{a'} Q(S_6, a') = -1.0 + (0.90 \times 5.00) = -1.0 + 4.50 = \mathbf{+3.50}$$

   El Error TD es intensamente positivo:

$$\delta = \text{Objetivo} - Q(S_3, \downarrow) = +3.50 - (-0.50) = \mathbf{+4.00}$$

   Actualizamos la celda $Q(S_3, \downarrow)$:

$$Q_{\text{nuevo}}(S_3, \downarrow) = -0.50 + 0.50 \times (+4.00) = -0.50 + 2.00 = \mathbf{+1.50}$$

4. **Paso 4 ($S_6 \to S_9$, Consolidación en la Meta):** En S_6, la acción Sur vale $+5.00$ (la más alta). OptiBot avanza al Sur hacia la Meta S_9 recibiendo $R_4 = +10.0$:

$$\text{Objetivo} = +10.0 + 0 = +10.00, \quad \delta = +10.00 - 5.00 = +5.00$$

$$Q_{\text{nuevo}}(S_6, \downarrow) = 5.00 + 0.50 \times (+5.00) = \mathbf{+7.50}$$

Al finalizar el tercer episodio, la Tabla Q refleja con claridad la propagación de la recompensa:

Estado (Casilla)	Norte (↑)	Sur (↓)	Este (→)	Oeste (←)
S_1 (Inicio)	0.00	−0.50	**−0.75**	0.00
S_2 (Pasillo)	0.00	0.00	**−0.75**	0.00
S_3 (Pasillo)	0.00	**+1.50**	0.00	0.00
S_4 (Pasillo)	0.00	−0.50	0.00	0.00
S_5 (Muro)	—	—	—	—
S_6 (Pasillo)	0.00	**+7.50**	0.00	0.00
S_7 (Pasillo)	0.00	0.00	−0.50	0.00
S_8 (Pasillo)	0.00	0.00	+5.00	0.00
S_9 (Meta)	0.00	0.00	0.00	0.00

Nota explicativa: Se observa con precisión matemática cómo la recompensa retrocede casilla por casilla: la antesala de la meta $(S_6, \downarrow)$ sube a $+7.50$ y la casilla previa $(S_3, \downarrow)$ salta por primera vez a un valor fuertemente positivo $(+1.50)$.

La intuición de las migajas de pan y el optimismo ante la incertidumbre: Este proceso ilustra dos pilares esenciales de Q-Learning:

1. **Exploración por optimismo inicial:** En el Episodio 2, el robot no repitió a ciegas la ruta norte porque su penalización la redujo a -0.50, mientras que las opciones no visitadas ofrecían 0.00. Esto empujó al agente a descubrir la ruta alternativa sur.
2. **Propagación hacia atrás de la recompensa:** La señal masiva de la Meta $(+10)$ retrocede un paso en cada recorrido completo. Primero iluminó la casilla vecina S_6 con $+5.00$, y en la siguiente visita iluminó la casilla previa S_3 con $+1.50$. Conforme avanzan los episodios, esta ola positiva llega a la casilla inicial (S_1), consolidando la política óptima.

1.7 Evolución a Largo Plazo: Cómo los Ceros se Transforman en Valores Negativos

Si las acciones transitables acumulan penalizaciones iniciales $(-0.50, -0.75)$, ¿por qué el robot no prefiere indefinidamente las acciones que aún valen 0.00?

La respuesta es que esos ceros reflejan únicamente falta de experiencia. Conforme el agente explora en episodios sucesivos $(\varepsilon \approx 1.0)$, descubre rápidamente el costo real de las malas decisiones:

1. **Salirse del laberinto (Game Over):** Al intentar cruzar los bordes perimetrales de la cuadrícula, el agente recibe una penalización inmediata de $R = -10.0$ y el episodio concluye (*Game Over*). Al ser un estado terminal sin futuro $(\max_{a'} Q = 0)$, la regla de actualización reduce drásticamente su valor:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[-10.0 + \gamma(0) - Q(s,a)\Big]$$

   En la primera visita con $\alpha = 0.50$, la celda cae de 0.00 a $\mathbf{-5.00}$, y en pocas iteraciones converge a $\mathbf{-10.00}$. Los ceros engañosos de los bordes se transforman así en valores fuertemente negativos.

2. **Chocar contra el muro central (S_5):** Intentar ingresar en la estantería bloqueada hace rebotar al robot en su misma casilla, sumando penalizaciones de paso $(R = -1.0)$ sin acercarlo a la meta.

1.7.1 El auge de los pasillos interiores: Descubrimiento de dos rutas óptimas

Mientras que las salidas prohibidas caen al fondo de la escala (≈ -10.00), los pasillos transitables experimentan el efecto contrario: aunque cada movimiento descuenta -1.00, están conectados a la enorme recompensa de $+10.0$ de la Meta (S_9).

A medida que el robot completa cientos de episodios, la recompensa positiva retrocede paso a paso hasta alcanzar el punto de partida (S_1). Al converger el entrenamiento, la Tabla Q queda completamente estructurada:

Estado (Casilla)	Norte (↑)	Sur (↓)	Este (→)	Oeste (←)
S_1 (Inicio)	-10.00	$\mathbf{+4.58}$	$\mathbf{+4.58}$	-10.00
S_2 (Pasillo)	-10.00	$+4.58$	$\mathbf{+6.20}$	$+3.12$
S_3 (Pasillo)	-10.00	$\mathbf{+8.00}$	-10.00	$+4.58$
S_4 (Pasillo)	$+3.12$	$\mathbf{+6.20}$	$+4.58$	-10.00
S_5 (Muro)	—	—	—	—
S_6 (Pasillo)	$+6.20$	$\mathbf{+10.00}$	-10.00	$+8.00$
S_7 (Pasillo)	$+4.58$	-10.00	$\mathbf{+8.00}$	-10.00
S_8 (Pasillo)	$+8.00$	-10.00	$\mathbf{+10.00}$	$+6.20$
S_9 (Meta)	0.00	0.00	0.00	0.00

Interpretación de la convergencia final:

- Las acciones que intentan abandonar el laberinto se han fijado en $\mathbf{-10.00}$, por lo que una política codiciosa jamás las seleccionará.
- En el inicio (S_1), el agente descubre que existen **dos caminos óptimos y simétricos** de 4 pasos con el mismo valor óptimo ($Q^* = +4.58$):
  - **Camino Superior:** $S_1 \xrightarrow{\text{Este}} S_2 \xrightarrow{\text{Este}} S_3 \xrightarrow{\text{Sur}} S_6 \xrightarrow{\text{Sur}} S_9$.
  - **Camino Inferior:** $S_1 \xrightarrow{\text{Sur}} S_4 \xrightarrow{\text{Sur}} S_7 \xrightarrow{\text{Este}} S_8 \xrightarrow{\text{Este}} S_9$.

1.8 Visualización Gráfica: La Política Aprendida en la Cuadrícula

Una vez que el algoritmo converge, ¿cómo se interpreta la Tabla Q en el espacio físico del almacén?

Para cada casilla s, simplemente dibujamos una flecha apuntando hacia la acción con el valor Q más elevado. La Figura 1.2 muestra el resultado:

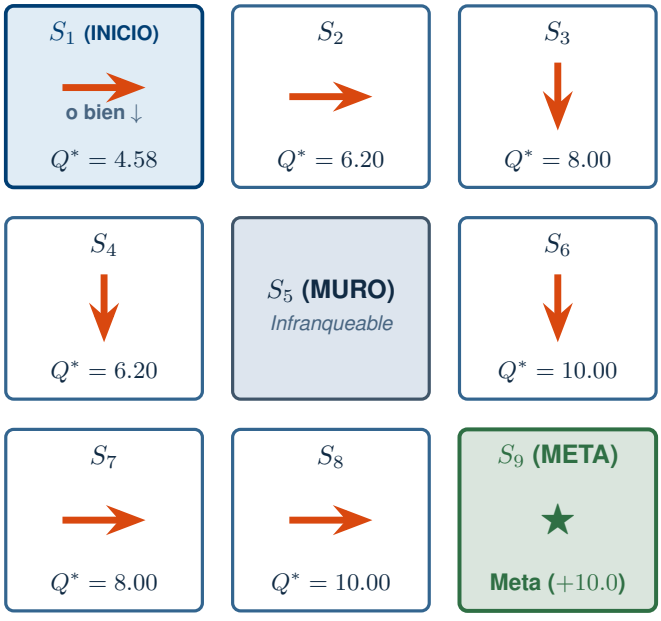


Figura 1.2: Política de navegación óptima aprendida por Q-Learning sobre la cuadrícula. Cada flecha en color coral indica la acción óptima $\pi^*(s) = \arg\max_a Q^*(s,a)$ en dicho estado, y el valor inferior muestra $Q^*(s, \pi^*(s))$. Obsérvese cómo las flechas guían inequívocamente a OptiBot bordeando el obstáculo central (S_5) a través de dos rutas mínimas de 4 pasos hacia la Meta (S_9).

1.9 Implementación en Python: El Laberinto de OptiBot

Para experimentar de forma interactiva con el entorno de simulación y el algoritmo tabular de Q-Learning, el código completo se encuentra disponible en el cuaderno Jupyter:

`notebooks/cap04_rl_tradicional.ipynb`

En este cuaderno interactivo se implementa la clase modular `LaberintoOptiBot` con la cuadrícula de 3×3, penalizaciones de frontera (-10.0 con *Game Over*) y recompensa por meta ($+10.0$). Asimismo, se programa el agente tabular `AgenteQLearning` con decaimiento de exploración ε, visualizando en tiempo real la convergencia de la tabla Q, las curvas de recompensa acumulada y la política direccional aprendida que conduce a la meta bordeando el obstáculo central por ambas rutas simétricas.

1.10 La Gran Limitación: La Maldición de la Dimensionalidad

La cuadrícula de 3×3 tiene apenas 9 estados y 4 acciones, por lo que la Tabla Q cabe cómodamente en cualquier hoja de cálculo con 36 celdas.

Sin embargo, ¿qué ocurriría si OptiBot saliera de la cuadrícula discreta de juguete y operara en un almacén real con sensores continuos?

1. **Sensores continuos:** Un sensor mide distancias con infinitos decimales ($d \in [0.05, 15.0]$ m), su velocímetro mide velocidades reales ($v \in [0.0, 2.5]$ m/s) y su batería decrece de forma continua ($b \in [0.0, 100.0]\,\%$).
2. **Espacio infinito de estados:** El número de combinaciones posibles en un entorno continuo es literalmente **infinito** ($|\mathcal{S}| = \infty$).
3. **El límite de las tablas:** No existe memoria RAM en el planeta que pueda almacenar una tabla de infinitas filas. Más aún, un robot jamás visitaría cada milímetro del espacio las veces requeridas para que cada celda aprenda.

Para sistemas reales, necesitamos sustituir la tabla rígida por un **aproximador continuo inteligente** capaz de generalizar: las **Redes Neuronales Profundas**.

El puente hacia el siguiente capítulo: En el Capítulo 2, daremos el gran salto conceptual hacia el **Aprendizaje por Refuerzo Profundo (Deep Q-Learning o DQN)**. Descubriremos cómo sustituir la Tabla Q por una red neuronal, por qué esta combinación puede ser inestable si no se controla adecuadamente (la *Tríada Mortal*), y cómo la arquitectura DQN de DeepMind logró estabilizar el aprendizaje mediante el buffer de repetición de experiencias y las redes objetivo congeladas.

2

Aprendizaje por Refuerzo Profundo: Deep Q-Learning (DQN)

2.1 De Tablas Discretas a Aproximadores de Funciones Continuos

En el Capítulo 1, constatamos la contundencia conceptual del algoritmo Q-Learning tabular, pero también su mayor limitación: la incapacidad intrínseca de escalar hacia entornos continuos o de alta dimensionalidad debido a la **maldición de la dimensionalidad**.

Cuando un agente interactúa con el mundo físico, sus sensores proporcionan flujos de información continuos. Para que un sistema de aprendizaje sea operativo en tales escenarios, no es viable memorizar de forma estanca cada punto del espacio de estados; el agente debe poseer la capacidad de **generalización inductiva**:

> *Si un robot aprende a evadir un obstáculo cuando su sensor LiDAR marca* 1.50 *m, debe ser capaz de inferir una maniobra evasiva similar cuando el sensor reporte* 1.52 *m, sin necesidad de haber experimentado exactamente esa lectura en el pasado.*

Para lograrlo, la tabla matricial $Q(s, a)$ se sustituye por una **función paramétrica diferenciable** implementada mediante una red neuronal artificial profunda:

$$Q(s, a; \boldsymbol{\theta}) \approx Q^*(s, a) \tag{2.1}$$

donde $\boldsymbol{\theta} \in \mathbb{R}^d$ agrupa todos los pesos sinápticos y sesgos (*biases*) ajustables de la red.

En esta etapa culminante, OptiBot deja atrás el tablero discreto y se conecta en tiempo real a su banco de telemetría sensorial continua. En cada milisegundo de muestreo, el procesador central del robot captura un vector de estado continuo de cuatro dimensiones $\mathbf{s} \in \mathbb{R}^4$:

1. $d_{\text{LiDAR}} \in [0.05, 15.0]$ m: Distancia continua al obstáculo más próximo medida por el escáner láser frontal.
2. $b_{\text{bat}} \in [0.0, 100.0]\,\%$: Nivel porcentual continuo de energía en los acumuladores de litio.
3. $v_{\text{act}} \in [0.0, 2.5]$ m/s: Velocidad lineal instantánea del vehículo en el pasillo.

4. $\theta_{\text{meta}} \in [-\pi, \pi]$ rad: Error angular de orientación respecto a la bahía de descarga.

Este vector continuo $\mathbf{s}$ se alimenta a la entrada de una red neuronal profunda, la cual produce simultáneamente a su salida una estimación continua del valor $Q(\mathbf{s}, a; \boldsymbol{\theta})$ para cada una de sus cuatro acciones discretas de movimiento:

$$\mathcal{A} = \{\text{Avanzar}, \text{GirarIzquierda}, \text{GirarDerecha}, \text{Frenar}\}$$

2.2 La Tríada Mortal: Por qué el RL Profundo Ingenuo Fracasa

Durante más de dos décadas, los investigadores intentaron sustituir directamente la tabla de Q-Learning por una red neuronal clásica, aplicando descenso de gradiente estándar tras cada paso. El resultado invariable fue el fracaso rotundo: las redes neuronales oscilaban caóticamente, los pesos explotaban hacia infinito o sufrían colapsos catastróficos de rendimiento.

Richard Sutton y Andrew Barto identificaron con precisión las tres condiciones concurrentes que provocan esta inestabilidad severa, bautizándolas colectivamente como la **Tríada Mortal**:

1. **Aproximador de funciones no lineal:** El uso de redes neuronales profundas con capas de activación no lineales (como ReLU o GELU).
2. **Bootstrapping:** La actualización de las estimaciones basándose en predicciones de la propia red en pasos temporales futuros ($R + \gamma \max Q$).
3. **Entrenamiento fuera de política (*Off-Policy*):** El aprendizaje de la política óptima codiciosa a partir de datos recolectados bajo una política exploratoria divergente.

2.2.1 Las Dos Patologías Destructivas del Aprendizaje

Cuando la Tríada Mortal se manifiesta en una arquitectura ingenua, desencadena dos patologías fatales:

1. Correlación temporal en las observaciones y olvido catastrófico

En el aprendizaje supervisado clásico, los algoritmos de optimización se fundamentan en el supuesto de que las muestras de entrenamiento son **independientes e idénticamente distribuidas** (i.i.d.).

En el aprendizaje por refuerzo, este supuesto se viola de manera flagrante: el robot experimenta secuencias temporales contiguas fuertemente correlacionadas (s_{t+1} depende directamente de s_t). Si OptiBot atraviesa un pasillo oscuro durante diez minutos, la red solo recibe datos de penumbra y ajusta sus pesos exclusivamente para ese entorno, sufriendo un **olvido catastrófico** de todo lo que había aprendido previamente en zonas iluminadas.

2. El problema del blanco móvil (*Moving Target*)

En la regla de actualización estándar de Q-Learning, el objetivo hacia el cual se ajusta la red es:

$$y_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \boldsymbol{\theta}) \tag{2.2}$$

¡Nótese que el objetivo depende directamente de los mismos parámetros $\boldsymbol{\theta}$ que se están intentando ajustar!

Cuando el optimizador modifica $\boldsymbol{\theta}$ para corregir la predicción en el estado actual S_t, altera involuntariamente las predicciones en el estado futuro S_{t+1}. Esto equivale a pedirle a un tirador

que acierte en una diana que se mueve a la misma velocidad que su propio brazo: el modelo persigue un blanco dinámicamente inestable, induciendo divergencia exponencial.

2.3 La Revolución de DQN: Los Dos Mecanismos Estabilizadores (DeepMind, 2015)

En 2015, Volodymyr Mnih y el equipo de Google DeepMind publicaron en la prestigiosa revista *Nature* el algoritmo **Deep Q-Network** (DQN). Lograron entrenar una única red neuronal profunda para jugar a más de 40 videojuegos clásicos de Atari 2600 directamente a partir de los píxeles de la pantalla, superando el nivel de jugadores humanos profesionales.

La genialidad de DQN radicó en neutralizar las patologías de la Tríada Mortal mediante la incorporación de dos mecanismos arquitectónicos revolucionarios:

2.3.1 1. Buffer de Repetición de Experiencias (*Experience Replay*)

En lugar de entrenar la red con la transición inmediata producida en cada paso y luego descartarla, el agente almacena cada una de sus vivencias en una memoria finita circular denominada **Replay Buffer** ($\mathcal{D}$):

$$e_t = \Big(s_t,\, a_t,\, r_{t+1},\, s_{t+1},\, \text{terminado}_{t+1}\Big) \in \mathcal{D} \tag{2.3}$$

Durante la fase de optimización, el algoritmo no entrena con la última transición; en su lugar, **extrae un mini-batch de transiciones muestreadas al azar de manera uniforme** de $\mathcal{D}$.

- **Ruptura de correlaciones temporales:** Al mezclar aleatoriamente transiciones de diferentes episodios y momentos temporales dispares, se restaura el régimen estadístico i.i.d.
- **Eficiencia muestral masiva:** Cada experiencia costosa recolectada en el entorno físico puede reutilizarse en decenas de actualizaciones de gradiente a lo largo del tiempo.

2.3.2 2. Red Objetivo Congelada (*Target Network*)

Para solucionar definitivamente el problema del blanco móvil, DQN desacopla el cálculo del objetivo manteniendo **dos conjuntos independientes de parámetros neuronales**:

1. **Red Online ($\boldsymbol{\theta}$):** La red neuronal principal cuyos pesos se actualizan continuamente en cada paso de gradiente. Es la red que decide las acciones del agente.
2. **Red Objetivo o Target Network ($\boldsymbol{\theta}^-$):** Una copia estructural idéntica cuyos parámetros se mantienen **estrictamente fijos y congelados** durante un período de C pasos temporales.

El objetivo de Bellman para cada muestra i del mini-batch se calcula evaluando exclusivamente los pesos congelados de la red target:

$$y_i = r_i + \gamma(1 - \text{terminado}_i) \max_{a' \in \mathcal{A}} Q(s'_i, a'; \boldsymbol{\theta}^-) \tag{2.4}$$

Dado que los parámetros $\boldsymbol{\theta}^-$ permanecen congelados, el objetivo y_i actúa como un valor constante y estacionario durante la optimización. Periódicamente (cada C pasos), los pesos de la red online se copian hacia la red objetivo ($\boldsymbol{\theta}^- \leftarrow \boldsymbol{\theta}$), o alternativamente se actualizan de forma suave mediante promedio de Polyak:

$$\boldsymbol{\theta}^- \leftarrow \tau\boldsymbol{\theta} + (1-\tau)\boldsymbol{\theta}^- \quad (\text{con } \tau \ll 1,\ \text{típicamente } \tau = 0.005) \tag{2.5}$$

2.4 Función de Pérdida y Descenso de Gradiente

Con el objetivo y_i estabilizado, el entrenamiento de la red online se formula como un problema de regresión supervisada, minimizando el **Error Cuadrático Medio de Bellman** (*Mean Squared Bellman Error*, o MSBE):

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{B}\sum_{i=1}^{B}\Big(y_i - Q(s_i, a_i; \boldsymbol{\theta})\Big)^2 \tag{2.6}$$

donde B denota el tamaño del mini-batch (habitualmente $B = 32$ o 64).

2.4.1 Pérdida de Huber (Smooth L1 Loss)

En presencia de transiciones con recompensas atípicas o discontinuidades abruptas, el error al cuadrado puede producir gradientes descomunales que desestabilicen los pesos de la red. Por ello, DQN sustituye comúnmente el error cuadrático por la **Pérdida de Huber**:

$$\mathcal{L}_{\text{Huber}}(\delta) = \begin{cases} \frac{1}{2}\delta^2 & \text{si } |\delta| \leq 1.0 \\ |\delta| - \frac{1}{2} & \text{en otro caso} \end{cases} \tag{2.7}$$

La función de Huber se comporta cuadráticamente para errores pequeños (facilitando una convergencia suave y precisa) y linealmente para errores grandes (impidiendo la explosión del gradiente).

2.4.2 El Enfoque del Semi-Gradiente

Al calcular la derivada de la pérdida respecto a los parámetros de la red online $\boldsymbol{\theta}$, el término objetivo y_i se trata formalmente como una constante externa fija:

$$\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}) = -\frac{2}{B}\sum_{i=1}^{B}\Big(y_i - Q(s_i, a_i; \boldsymbol{\theta})\Big)\nabla_{\boldsymbol{\theta}} Q(s_i, a_i; \boldsymbol{\theta}) \tag{2.8}$$

Este método se denomina **semi-gradiente** porque omite deliberadamente el efecto que $\boldsymbol{\theta}$ tendría sobre el objetivo futuro (el cual se encuentra congelado en $\boldsymbol{\theta}^-$).

2.5 Cálculo Numérico Paso a Paso: Flujo de Tensores en OptiBot

Para desmitificar el funcionamiento interno de una red neuronal en RL, sigamos minuciosamente el cálculo numérico para una muestra extraída del Replay Buffer de OptiBot:

Considérese una transición extraída aleatoriamente de la memoria de repetición con hiperparámetros: factor de descuento $\gamma = 0.95$ y tasa de aprendizaje $\eta = 0.001$.
Datos de la transición extraída:

- Estado inicial continuo: $\mathbf{s} = [1.20\text{ m}, 85.0\,\%, 1.50\text{ m/s}, 0.10\text{ rad}]^T$.
- Acción ejecutada en ese instante: $a = 0$ (*Avanzar*).
- Recompensa recibida del entorno: $r = +1.50$.
- Estado sucesor alcanzado: $\mathbf{s}' = [0.80\text{ m}, 84.8\,\%, 1.60\text{ m/s}, 0.05\text{ rad}]^T$.
- Bandera de episodio terminal: terminado $=$ Falso (0).

Paso 1: Paso hacia adelante en la Red Target con $\mathbf{s}'$ El vector de estado futuro $\mathbf{s}'$ se propaga por la red congelada $Q(\cdot;\boldsymbol{\theta}^-)$, obteniéndose las siguientes predicciones para cada acción admisible:

Acción	$a'=0$ (Avanzar)	$a'=1$ (GirarIzq)	$a'=2$ (GirarDer)	$a'=3$ (Frenar)
$Q(\mathbf{s}',a';\boldsymbol{\theta}^-)$	4.20	6.50	3.80	1.10

La mejor opción futura predicha por la red target es $a'=1$ con valor:

$$\max_{a'\in\mathcal{A}} Q(\mathbf{s}',a';\boldsymbol{\theta}^-) = \mathbf{6.50}$$

Paso 2: Cómputo del Objetivo Bellman Congelado (y) Aplicando la ecuación (2.4):

$$\begin{aligned} y &= r + \gamma(1-\text{terminado})\max_{a'} Q(\mathbf{s}',a';\boldsymbol{\theta}^-) \\ &= 1.50 + (0.95\times 1\times 6.50) \\ &= 1.50 + 6.175 = \mathbf{7.675} \end{aligned}$$

Paso 3: Paso hacia adelante en la Red Online con $\mathbf{s}$ Se evalúa el estado inicial $\mathbf{s}$ en la red online activa $Q(\cdot;\boldsymbol{\theta})$:

$$Q(\mathbf{s},a=0;\boldsymbol{\theta}) = \mathbf{5.100}$$

Paso 4: Determinación del Error TD (δ) y Pérdida Cuadrática El error entre la predicción activa y el objetivo fijado resulta:

$$\delta = y - Q(\mathbf{s},a=0;\boldsymbol{\theta}) = 7.675 - 5.100 = +\mathbf{2.575}$$

La pérdida cuadrática correspondiente para este ejemplo es:

$$\mathcal{L} = \delta^2 = (2.575)^2 \approx \mathbf{6.63}$$

Paso 5: Retropropagación y Actualización de Parámetros El algoritmo calcula el gradiente $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ mediante diferenciación automática (*Backpropagation*). El optimizador (como Adam) ajusta los pesos de la red online en dirección opuesta al gradiente, incrementando el valor de salida $Q(\mathbf{s},a=0;\boldsymbol{\theta})$ para aproximarlo a 7.675.
Mientras tanto, los parámetros de la red target $\boldsymbol{\theta}^-$ permanecen completamente inmutables, garantizando una optimización sin oscilaciones.

2.6 Double DQN (DDQN): Eliminación del Sesgo de Sobreestimación

A pesar de su éxito, el algoritmo DQN estándar padece una imperfección matemática inherente: **el sesgo sistemático de sobreestimación** inducido por el operador $\max_{a'}$.

En presencia de ruido estocástico o imprecisiones de aproximación de la red, la esperanza matemática del máximo entre variables aleatorias siempre es mayor o igual que el máximo de sus esperanzas:

$$\mathbb{E}\left[\max_a X_a\right] \geq \max_a \mathbb{E}[X_a] \tag{2.9}$$

Dado que DQN utiliza el mismo conjunto de pesos $\boldsymbol{\theta}^-$ tanto para **seleccionar** cuál es la mejor acción como para **evaluar** su valor, tiende a sobreestimar sistemáticamente los valores Q, lo que puede degradar la política en entornos complejos.

Para erradicar este problema, Hado van Hasselt et al. (2016) introdujeron **Double DQN** (DDQN), desacoplando completamente ambos roles:

$$y_i^{\text{DDQN}} = r_i + \gamma(1 - \text{terminado}_i) Q\left(s'_i, \underbrace{\arg\max_{a' \in \mathcal{A}} Q(s'_i, a'; \boldsymbol{\theta})}_{\text{Selección con Red Online}}; \underbrace{\boldsymbol{\theta}^-}_{\text{Evaluación con Red Target}} \right) \tag{2.10}$$

- La **Red Online** ($\boldsymbol{\theta}$) decide qué acción es la mejor candidata.
- La **Red Target** ($\boldsymbol{\theta}^-$) calcula de manera imparcial el valor Q atribuible a esa acción elegida.

Si una acción fue sobreestimada por azar en la red target, es improbable que la red online coincida en seleccionarla, cancelando de raíz el sesgo positivo sistemático.

2.7 Implementación Modular en PyTorch: OptiBot DQN

Para experimentar de forma interactiva con el entrenamiento de redes neuronales profundas para control por refuerzo, el código completo se encuentra disponible en el cuaderno Jupyter:

`notebooks/cap05_rl_profundo.ipynb`

En este cuaderno interactivo se implementa la arquitectura modular en PyTorch: la red neuronal de aproximación de valor `RedQOptiBot`, el búfer de repetición de experiencias `BufferReplay` (descorrelación de muestras temporales) y el agente `AgenteDQN` integrando *Double DQN* y actualización suave de pesos (*Polyak averaging* con parámetro τ). El cuaderno incluye bucles de entrenamiento en entornos continuos, seguimiento de la pérdida de Huber y curvas de convergencia del retorno acumulado.

2.8 Comparativa Global: De las Tablas a las Redes Neuronales

La Tabla 2.1 resume la transición paradigmática recorrida a lo largo de este curso, contrastando las dos grandes eras del Aprendizaje por Refuerzo:

Dimensión	RL Tradicional (Q-Learning)	RL Profundo (DQN)
Representación	Matriz o tabla discreta $Q(s, a)$.	Red Neuronal Profunda paramétrica $Q(s, a; \boldsymbol{\theta})$.
Espacio de Estados	Finito y reducido (pocos estados discretos).	Continuo, sensorial y de alta dimensionalidad ($\mathbb{R}^d$, imágenes).
Generalización	Nula: cada celda (s, a) debe visitarse de forma individual.	Total: interpola comportamientos óptimos para estados jamás observados.
Garantías Teóricas	Convergencia asintótica garantizada con probabilidad 1 (Watkins).	Convergencia empírica; sujeto a la Tríada Mortal si no se estabiliza.
Estabilizadores Clave	Tasa de aprendizaje decreciente α_t y política ε-greedy.	Buffer de Repetición de Experiencias y Red Objetivo Congelada.

Cuadro 2.1: Comparativa estructural entre el Aprendizaje por Refuerzo Tradicional y Profundo.

Conclusión del Curso y Perspectivas Futuras: A lo largo de este manual, hemos construido de forma progresiva, rigurosa y transparente el andamiaje completo del Aprendizaje por Refuerzo:

- Partimos de la cuantificación de la incertidumbre probabilística y los bandidos multibrazo.
- Formalizamos las transiciones dinámicas mediante Cadenas y Procesos de Decisión de Markov, deduciendo analíticamente las Ecuaciones de Bellman.
- Desarrollamos el control libre de modelo mediante Diferencia Temporal y Q-Learning tradicional.
- Finalmente, conquistamos los espacios continuos mediante el Aprendizaje por Refuerzo Profundo y las redes DQN.

Con estas bases matemáticas y computacionales, el estudiante se encuentra plenamente capacitado para abordar las fronteras avanzadas del aprendizaje autónomo, tales como los métodos de Gradiente de Política (*Policy Gradients*, PPO, SAC) y los sistemas modernos de alineamiento inteligente.